

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

1. Introduction:

This System Analysis Document (SAD) outlines the technical architecture, design decisions, data flows, and implementation strategy for the Jalan-Ready system submitted to UMHackathon 2026 under the AI Systems & Agentic Workflow Automation category.

1.1. Purpose:

This document serves as a technical reference for developers, testers, and judges to understand:

- *The multi-agent LangGraph architecture with Z.AI GLM as the central reasoning engine*
- *Data flows from citizen input to authority dispatch*
- *Model selection justification and prompting strategies*
- *State management and workflow orchestration*
- *Deployment architecture on Streamlit Community Cloud*

1.2. Background:

Selangor's road infrastructure management is fragmented across multiple authorities: JKR Federal, JKR State (9 district offices), and 11 Local Councils (PBTs). Citizens often report issues to the wrong department, causing delays. Existing systems use rigid scripts that fail when faced with ambiguity (missing GPS, unclear descriptions) or dynamic conditions (weather, traffic, recurrence patterns).

Jalan-Ready replaces scripted workflows with an agentic GLM-powered orchestrator that reasons about each report, handles missing data conversationally, adapts to weather conditions, detects recurrence patterns, and autonomously routes to the correct Selangor authority.

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
Citizens (Public)	Report road defects via chat + photo upload	Simple conversational interface, clear priority display, trackable status
JKR District Officers	Receive work orders for Federal/State roads within their district	Correct jurisdiction routing, complete assessment data, recurrence alerts

<i>Local Council Officers (PBT)</i>	<i>Receive work orders for municipal/city roads</i>	<i>Accurate mapping of report to their administrative boundary</i>
<i>Contractors</i>	<i>Execute repairs based on optimized daily routes</i>	<i>Traffic-aware scheduling, multi-stop route optimization</i>
<i>QA Team</i>	<i>Validating the system behavior for all the user roles</i>	<i>Defined scope, the request and response formats, and coverage of edge cases</i>
<i>Development Team</i>	<i>Build and maintain the platform</i>	<i>Clear architecture, modular agents, documented APIs</i>

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web application (Streamlit)
Architecture	Multi-Agent LangGraph workflow with GLM as central orchestrator
Deployment	Streamlit Community Cloud (public demo/pilot)
Concurrent Users	5 simultaneous users (pilot scale)

Jalan- Ready is structured as a client-server system with:

- Frontend: Streamlit web application (user chat, dashboard, report submission)
- Backend Agents: 4 specialized LangGraph agents (Input, Damage, Scheduling, Dispatch)
- Central Reasoner: Z.AI GLM (entity extraction, jurisdiction reasoning, priority calculation)
- Vision Module: YOLOv8 ONNX (local execution for defect detection)
- Database: SQLite (stateful tracking of reports)
- External APIs: Open-Meteo (weather), Google Maps Geocoding (location → jurisdiction mapping)

2.1.2 GLM as Service Layer

The Z.AI GLM is NOT a generic "AI module" but the central reasoning orchestrator. The architecture includes:

2.1.3 Dependency Diagram

- a. How the prompts are being constructed and sent to the GLM API
- b. What goes into the context window?
- c. How is your system receiving, parsing the responses and passing to the next system component?
- d. Where the limitation of token are enforced or input are chunked before it actually reaches to GLM

- e. Also, do outline all the major API calls between the system components [e.g. Customer Mobile App calls "Amazon API Gateway" for Order Service]
- f. Do show how frontend, backend, services, database and external API interacts with each other [e.g. Order service communicates with Amazon RDS to ensure persistence of order data]
- g. Must highlights the distributed components if the system is planning to be deployed across multiple services

2.2.2. Sequence Diagram

- a. Here, do walk through one user interaction flow or customer journey with the stakeholders [e.g. Place Order Flow]
- b. Show the interactions step by step.
- c. Do demonstration of each key system features does operate from start to finish

2.2. Technological Stack

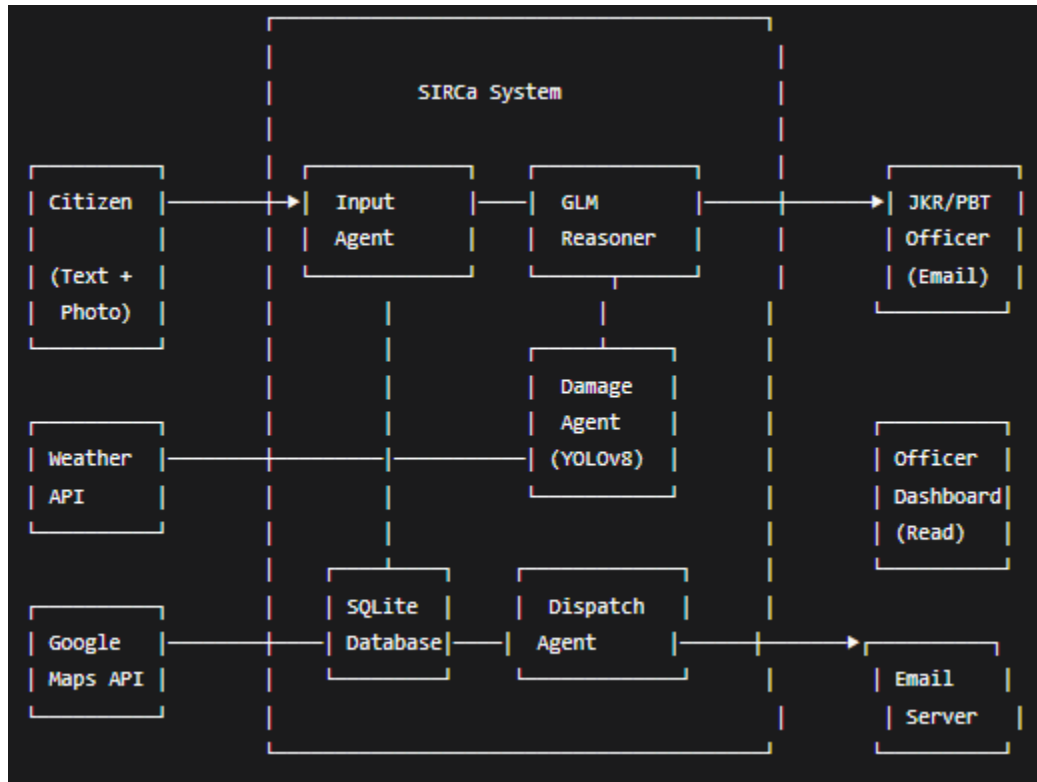
Frontend	Streamlit	Rapid prototyping, built-in chat components, easy deployment to Community Cloud, Python-native
Backend / Agents	LangGraph + Python 3.10	Stateful workflow management, checkpoint support for resumable workflows, native function calling
GLM Integration	Z.AI GLM API (via OpenAI-compatible SDK)	Required by hackathon; serves as central reasoning orchestrator
Vision Model	YOLOv8 (converted to ONNX format)	Lightweight, local execution (no GPU needed), real-time detection
Database	SQLite (file-based)	Zero-configuration, sufficient for pilot with 5 concurrent users
External APIs	Open-Meteo (weather), Google Maps Geocoding (location → address)	Free tier available for pilot; reliable and well-documented
Email	SMTP (smtplib)	Native Python, works with any email provider
Deployment	Streamlit Community Cloud	Free hosting, GitHub integration, environment

variable support

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)



Data Movement Validation (No Black Holes):

- All data that enters the system either: (a) persists in SQLite, (b) transforms into an email output, or (c) returns to the user as a response
- No external entity sends data that disappears without processing
- Input validation ensures malformed data triggers `AWAITING_INFO` state rather than silent discard

2.3.2. Normalized Database Schema

- Show the logical data model (ERD) with Database designed to 3NF form to ensure data integrity and relationships. [e.g. order items is a junction

table that is resolving the M: N relationship between orders and menu items.

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

3.1.1. Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026. [e.g. in Table]

#	Features	Description
1	Conversational Report Submission	Users describe road defects in natural language + upload photos; system responds conversationally
2	YOLOv8 Vision Analysis	Real-time detection of potholes, cracks, faded lines, standing water with confidence scores
3	GLM-Powered Jurisdiction Routing	System determines if road is Federal/State (→ JKR District) vs Municipal/City (→ Local Council) based on reverse geocoding
4	Dynamic Priority Calculation	GLM calculates urgency score (1-100) based on severity, road class, proximity to critical infrastructure, recurrence flag
5	Stateful Workflow with SQLite	Tracks reports through NEW → AWAITING_INFO → REPORTED → IN_PROGRESS → ESCALATED → RESOLVED
6	Weather-Aware Scheduling	Checks past 72h rainfall to flag sub-base saturation; checks 48h forecast to avoid scheduling during rain
7	Recurrence Detection	Queries database for similar reports within 100m radius, last 90 days; escalates priority if ≥2 past reports
8	Email Dispatch to Authorities	Sends structured work order to correct JKR District or Local Council email address
9	User Dashboard	Displays all user reports with status and priority
10	LangGraph Orchestration	Multi-agent workflow with checkpoint support for resumable conversations

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g.)
Scalability	System must handle 5 concurrent users on Streamlit Cloud.	Streamlit Cloud auto-scales; SQLite file-based with row-level locking sufficient for pilot.
Reliability	Reports must not be lost on API failure; 99% workflow completion rate.	Retry queue with exponential backoff (5min, 30min, 2hr) for email failures; GLM API timeout set to 10s with fallback to manual review.
Maintainability	Each agent (Input, Damage, Scheduling, Dispatch) separated and independently testable.	LangGraph modular architecture; independent unit tests per agent; CI/CD via GitHub Actions.
Token Latency	GLM API calls must return within 5 seconds under normal load (p95).	Timeout 10s; user-facing loading state; truncation of long conversation history to last 5 exchanges.
Cost Efficiency	Average token cost per user session must not exceed 1,000 tokens.	Fixed-length prompts; truncation of long inputs; caching of jurisdiction mapping (24hr TTL).
Privacy	User data (photos, location, reports) must not be exposed to unauthorized parties.	Streamlit authentication (simple password); no PII stored beyond email; photos stored with random filenames.
Local Execution	Vision model must run locally without GPU dependency.	YOLOv8 converted to ONNX format; runs on CPU with 200-500ms inference.

3.3. Out of Scope / Future Enhancements

Do identify the high-level features that the team planned for “Grand Vision” but not part of this UMHackathon 2026. [e.g. below]

- a. During delivery, the In-App chat features between the rider and customer
- b. Promotional offers, discount code, vouchers
- c. Scheduled Delivery (Order now, and wants delivery to be delivered on a specific time)

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

- MVP deployed to 5% of pilot users initially (internal testers)
- Monitoring for 48 hours; check for error rate <5% and GLM response validity >80%
- Gradual increase to 25%, then 50%, then 100% if no issues detected
- A/B testing for prompt strategies: Compare zero-shot vs few-shot GLM prompts for jurisdiction accuracy

4.1.2. Emergency Rollback (ER) & Golden Release:

- Golden release tagged in GitHub (e.g., v1.0.0-golden)
- Emergency rollback triggered if: GLM API error rate >10%, email dispatch failure rate >5%, or jurisdiction misrouting >15%
- Rollback executed via git revert + redeploy to Streamlit Cloud (<5 minutes)

4.1.3. Priority Matrix:

Priority	Trigger Condition	Action
P0 (Critical)	System crashes or 500 error on any endpoint	Immediate rollback to golden release; notify team
P1 (High)	GLM API error rate >10% over 5 minutes	Fallback to simplified prompt; cache last valid response
P2 (Medium)	Email dispatch failure rate	Switch to backup SMTP

	>5% over 30 minutes	server; queue failed emails
P3 (Low)	YOLOv8 inference >1 second per image	Log performance warning; no user-facing impact

4.2. **Monitoring:**

<i>Metric</i>	<i>Threshold</i>	<i>Alert Action</i>
<i>GLM API response time</i>	<i>>5 seconds (p95)</i>	<i>Log warning; investigate API performance</i>
<i>GLM JSON parse failures</i>	<i>>10% of requests</i>	<i>Fallback to retry; notify developer</i>
<i>Email dispatch success rate</i>	<i><95%</i>	<i>Queue for retry; switch to backup SMTP</i>
<i>Report completion rate (NEW → REPORTED)</i>	<i><80%</i>	<i>Check GLM prompt quality; review missing data handling</i>
<i>Database query time</i>	<i>>500ms</i>	<i>Optimize indexes; consider archiving old reports</i>
<i>YOLOv8 inference time</i>	<i>>1 second</i>	<i>Log performance; consider model quantization</i>

4.3. **Assumptions:**

- Users have stable internet connections when accessing Streamlit app
- JKR and PBT email addresses (aduan@...) are correctly formatted and monitored by the authorities
- Open-Meteo API remains available for weather data (free tier, no API key required)
- Google Maps Geocoding API free tier (200 requests/month) is sufficient for pilot with 5 concurrent users
- Streamlit Community Cloud free tier supports 5 concurrent users without performance degradation
- Citizens are comfortable providing location data (GPS from browser or manual address input)

- YOLOv8 ONNX model (trained on pothole dataset) generalizes to other defect types (cracks, faded lines) for demo purposes

4.4. **External Dependencies:** Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks
Z Ai GLM API	Central reasoning orchestrator	API downtime, rate limits, cost
Open-Meteo API	Historical + forecast weather data	API downtime, data unavailability
SMTP (email provider)	Dispatch work orders	Email server rejection, rate limits
Streamlit Community Cloud	Hosting	Deployment limits, session timeouts
Google Maps API	Reverse geocoding, jurisdiction mapping	Rate limits (200 requests/month free tier)

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

Day 1-2: Environment Setup & Database Initialization (Python environment, SQLite schema, jurisdiction mapping table)

Day 3-4: Vision Module (YOLOv8) (Train/demo detection model, ONNX export, image preprocessing)

Day 5-6 : LangGraph Workflow & GLM Integration (4 agents, GLM API connection, JSON schema validation)

Day 7-8 : Streamlit UI Development (Chat interface, user dashboard, report history)

Day 9 : Testing & Edge Cases (Happy path, missing data, API failures, recurrence detection)

Day 10 : Deployment & Documentation (GitHub push, Streamlit Cloud deployment, final PRD/SAD/QATD)

5.2. Team Members Role:

© UMHackathon 2026

Email: umhackathon@um.edu.my

Backend Lead : *LangGraph workflow, GLM integration, SQLite schema, API orchestration*
Frontend Lead: *Streamlit UI, chat components, dashboard, user authentication*
ML/Vision Engineer: *YOLOv8 training, ONNX conversion, image preprocessing, detection pipeline*
QA & Testing Lead: *Test case creation, CI/CD pipeline, edge case validation, GLM output testing*

5.3. **Recommendations:**

<i>Cache Google Maps Geocoding results for 24 hours</i>	<i>Reduces API calls; free tier limited to 200 requests/month</i>
<i>Implement GLM response validation layer</i>	<i>Catches hallucinated jurisdictions or invalid priority scores before dispatch</i>
<i>Use environment variables for all API keys (Streamlit secrets)</i>	<i>Prevents key exposure in GitHub repository</i>
<i>Add request rate limiting per user session</i>	<i>Prevents abuse on public Streamlit deployment</i>
<i>Implement async email dispatch with queue</i>	<i>Prevents UI blocking during email sending; enables retries</i>
<i>Log all GLM requests/responses to file</i>	<i>Facilitates debugging and prompt engineering improvements</i>